Finally, when the max is reached, the actor stops by setting the next behavior as `Behaviors.stopped`.

NOTE Once the actor stops, it cannot process any more messages. All messages remaining in its queue or new messages sent to its address are forwarded by the system to another actor named `deadLetters` (more about this in chapter 3). Bear in mind that `apply()` is not a recursive function but a factory that receives a function as input and outputs a `Behavior` (a function). The factory `Behaviors.receive` that defines the `apply` method receives a function of type `(context, message) => Behavior` and therefore outputs a `Behavior`. This is functional style, and there is no recursion.

Let's look at the wallet, its logging, and how it manages state when it receives `Increase`.

Listing 2.5 `Wallet` actor with state

```
import akka.actor.typed.Behavior
import akka.actor.typed.scaladsl.Behaviors

object WalletState {

  sealed trait Command
  final case class Increase(amount: Int) extends Command
  final case class Decrease(amount: Int) extends Command

  def apply(total: Int, max: Int): Behavior[Command] =    ❶
    Behaviors.receive { (context, message) =>
      message match {
        case Increase(amount) =>
          val current = total + amount
          if (current <= max) {
            context.log.info(s"increasing to $current")
            apply(current, max)                           ❷
          } else {
```